

PRACTICE SHEET – 3 (Aryan Kushwaha – 245703428)

Q1: // Remove duplicates in-place from a sorted vector

```
#include <iostream>
using namespace std;

void insertElement(int arr[], int& size, int pos, int val, int capacity) {
    if (pos < 0 || pos > size || size >= capacity) {
        cout << "Invalid position or array is full.\n";
        return;
    }
    for (int i = size; i > pos; --i) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = val;
    size++;
}

void deleteElement(int arr[], int& size, int pos) {
    if (pos < 0 || pos >= size) {
        cout << "Invalid position.\n";
        return;
    }
    for (int i = pos; i < size - 1; ++i) {
        arr[i] = arr[i + 1];
    }
    size--;
}

int getLength(int size) {
    return size;
}

int main() {
    int arr[100];
    int size = 0;
    int capacity = 100;

    insertElement(arr, size, 0, 10, capacity);
    insertElement(arr, size, 1, 20, capacity);
    insertElement(arr, size, 1, 15, capacity);

    cout << "Array after insertions: ";
    for (int i = 0; i < size; ++i) cout << arr[i] << " ";
    cout << "\nLength: " << getLength(size) << "\n";

    deleteElement(arr, size, 1);

    cout << "Array after deletion at index 1: ";
    for (int i = 0; i < size; ++i) cout << arr[i] << " ";
    cout << "\nLength: " << getLength(size) << "\n";

    return 0;
}
```

```
}
```

Output:

```
Array after insertions: 10 15 20
Length: 3
Array after deletion at index 1: 10 20
Length: 2
```

```
2. // Find the common elements between two vectors
```

```
#include <iostream>
#include <vector>
using namespace std;

bool isPresent(const vector<int>& vec, int element) {
    for (int i = 0; i < vec.size(); ++i) {
        if (vec[i] == element) {
            return true;
        }
    }
    return false;
}

vector<int> findCommonElements(const vector<int>& v1, const vector<int>& v2) {
    vector<int> common;

    for (int i = 0; i < v1.size(); ++i) {
        for (int j = 0; j < v2.size(); ++j) {
            if (v1[i] == v2[j]) {
                if (!isPresent(common, v1[i])) {
                    common.push_back(v1[i]);
                }
                break;
            }
        }
    }
    return common;
}

int main() {
    vector<int> vec1 = {1, 4, 5, 9, 2, 4};
    vector<int> vec2 = {9, 2, 4, 8, 10};

    vector<int> common = findCommonElements(vec1, vec2);

    cout << "Common elements: ";
    for (int i = 0; i < common.size(); ++i) {
        cout << common[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

```
}
```

Output:

```
Common elements: 4 9 2
```

Q3: // Split a vector into even and odd vectors

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> original = {12, 5, 8, 13, 22, 9, 1, 4, 10};
    vector<int> evens;
    vector<int> odds;

    for (int i = 0; i < original.size(); ++i) {
        if (original[i] % 2 == 0) {
            evens.push_back(original[i]);
        } else {
            odds.push_back(original[i]);
        }
    }

    cout << "Even numbers: ";
    for (int i = 0; i < evens.size(); ++i) {
        cout << evens[i] << " ";
    }
    cout << "\n";

    cout << "Odd numbers: ";
    for (int i = 0; i < odds.size(); ++i) {
        cout << odds[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

Output:

```
Even numbers: 12 8 22 4 10
Odd numbers: 5 13 9 1
```

Q4: // Count the occurrences of each element in a vector

```
#include <iostream>
#include <vector>
using namespace std;

void countFrequencies(const vector<int>& vec) {
    vector<bool> visited(vec.size(), false);

    for (int i = 0; i < vec.size(); ++i) {
        if (visited[i]) {
            continue;
        }

        int count = 1;
        for (int j = i + 1; j < vec.size(); ++j) {
            if (vec[i] == vec[j]) {
                visited[j] = true;
                count++;
            }
        }

        cout << "Element " << vec[i] << " occurs " << count << " time(s)\n";
    }
}

int main() {
    vector<int> vec = {5, 2, 5, 8, 2, 5, 1, 8, 9};

    countFrequencies(vec);

    return 0;
}
```

Output:

```
Element 5 occurs 3 time(s)
Element 2 occurs 2 time(s)
Element 8 occurs 2 time(s)
Element 1 occurs 1 time(s)
Element 9 occurs 1 time(s)
```